🎯 **Atividade Visibilidade Atributos**

📕 **Orientação de Estrutura:**

1. Crie um arquivo com o nome abaixo:
    - Pasta: 05_POO
    - Nome: atividade_**x** -> Onde x é o número da atividade

---

🧠 **Atividade individual**

👨‍🏫 **Consulta apenas do material do professor**

👨‍💻 **Atividade 8**

- Crie a classe **Funcionario** com os seguintes **atributos privados**:
    - Nome
    - Salário
- Crie um **método construtor público** que receba os parâmetros nome e salário.
    - Caso o salário não seja informado, ele deve assumir o valor padrão de **1000**.
- Crie o método **público** aumentarSalario($percentual) que aumente o salário do funcionário de acordo com o percentual informado.
    - O aumento só deve ser permitido se o percentual for maior que 0 e menor ou igual que 10.
    - Caso o percentual seja inválido, exiba uma mensagem de erro informando o intervalo permitido.
- Crie o método **público** exibirSalario() que exiba na tela o nome do funcionário e seu salário formatado em reais, no seguinte formato:

Exemplo de execução

```php
$func = new Funcionario("Carlos", 3000);
$func->aumentarSalario(10);
$func->exibirSalario();
```

Resultado no navegador

Funcionário: Carlos - Salário: R$ 3.300,00

Resposta

```php
<?php
class Funcionario {
    private $nome;
    private $salario;

    public function __construct($nome, $salario = 1000) {
        $this->nome = $nome;
        $this->salario = $salario;
    }

    public function aumentarSalario($percentual) {
        if ($percentual > 0 && $percentual <= 10) {
            $this->salario += $this->salario * ($percentual / 100);
        }else{
            echo "Percentual de aumento incorreto, o valor precisa estar entre 0 e 10";
        }
    }

    public function exibirSalario() {
        echo "Funcionário: $this->nome - Salário: R$ " .
            number_format($this->salario, 2, ',', '.') . "<br>";
    }
}

$func = new Funcionario("Carlos", 3000);
$func->aumentarSalario(10);
$func->exibirSalario();
```

👨‍💻 Atividade 9

- Crie a classe **Produto** com os seguintes **atributos privados**:
  - Nome
  - Preco
  - Estoque
- Crie um **método construtor público** que receba os parâmetros nome, preco e estoque, e atribua os valores aos atributos correspondentes.
- Crie o método **público** vender($quantidade) que permita realizar a venda de um número específico de unidades do produto.
  - Se a quantidade solicitada for menor ou igual ao estoque, o estoque deve ser reduzido e deve ser exibida uma mensagem de sucesso.
  - Caso contrário, deve ser exibida uma mensagem informando que o estoque é insuficiente.
- Crie o método **público** reajustarPreco($percentual) que permita alterar o preço do produto de acordo com o percentual informado.
- Crie o método **público** exibirInfo() que exiba as informações do produto.

Exemplo de execução

```php
$produto = new Produto("Teclado Gamer", 250, 10);
$produto->vender(2);
$produto->reajustarPreco(10);
$produto->exibirInfo();
```

Resultado no navegador

Venda de 2 unidade(s) de Teclado Gamer realizada!
Teclado Gamer → R$ 275,00 | Estoque: 8 unidades

Resposta

```php
<?php
class Produto {
    private $nome;
    private $preco;
    private $estoque;

    public function __construct($nome, $preco, $estoque) {
        $this->nome = $nome;
        $this->preco = $preco;
        $this->estoque = $estoque;
    }

    public function vender($quantidade) {
        if ($quantidade <= $this->estoque) {
            $this->estoque -= $quantidade;
            echo "Venda de $quantidade unidade(s) de $this->nome realizada!<br>";
        } else {
            echo "Estoque insuficiente para $this->nome!<br>";
        }
    }

    public function reajustarPreco($percentual) {
        $this->preco += $this->preco * ($percentual / 100);
    }

    public function exibirInfo() {
        echo "$this->nome → R$ " . number_format($this->preco, 2, ',', '.') .
             " | Estoque: {$this->estoque} unidades<br>";
    }
}

$produto = new Produto("Teclado Gamer", 250, 10);
$produto->vender(2);
$produto->reajustarPreco(10);
$produto->exibirInfo();
```

👨‍💻 **Atividade 10**

- Crie a classe **Carro** com os seguintes **atributos privados**:
  - Modelo
  - consumo (em km por litro)
  - tanque (litros disponíveis)
- Crie um **método construtor público** que receba os parâmetros modelo, consumo e tanqueInicial.
  - Caso consumo ou tanqueInicial não sejam informados, eles devem assumir os valores padrão de **10 km/L** e **0 litros**, respectivamente.
- Crie o método **público** abastecer($litros) que permita adicionar combustível ao tanque.
  - Só deve aceitar valores positivos e exibir uma mensagem indicando a quantidade abastecida.
  - Caso o valor seja inválido, exiba uma mensagem de erro.
- Crie o método **público** dirigir($km) que simule a viagem do carro.
  - O método deve calcular o combustível necessário para percorrer a distância informada e reduzir o nível do tanque.
  - Se não houver combustível suficiente, exiba uma mensagem informando a impossibilidade de percorrer a distância.
- Crie o método **público** exibirInfo() que exiba as informações do carro.

Exemplo de execução

```php
$carro = new Carro("Honda Civic", 12, 20); // 12 km/L, tanque inicial de 20 L
$carro->exibirInfo();
$carro->dirigir(60); // deve consumir 5 L
$carro->abastecer(10);
$carro->exibirInfo();
```

Resultado no navegador

Modelo: Honda Civic | Consumo: 12 km/L | Tanque: 20,00 L
O carro Honda Civic percorreu 60 km e consumiu 5,00 litros.
Abastecidos 10 litro(s) no Honda Civic.
Modelo: Honda Civic | Consumo: 12 km/L | Tanque: 25,00 L

Resposta

```php
<?php
class Carro {
    private $modelo;
    private $consumo; // km por litro
    private $tanque;  // litros disponíveis

    public function __construct($modelo, $consumo = 10, $tanqueInicial = 0) {
        $this->modelo = $modelo;
        $this->consumo = $consumo;
        $this->tanque = $tanqueInicial;
    }

    public function abastecer($litros) {
        if ($litros > 0) {
            $this->tanque += $litros;
            echo "Abastecidos $litros litro(s) no {$this->modelo}.<br>";
        } else {
            echo "Quantidade inválida para abastecimento!<br>";
        }
    }

    public function dirigir($km) {
        $litrosNecessarios = $km / $this->consumo;

        if ($litrosNecessarios <= $this->tanque) {
            $this->tanque -= $litrosNecessarios;
            echo "O carro {$this->modelo} percorreu $km km e consumiu " .
                 number_format($litrosNecessarios, 2, ',', '.') . " litros.<br>";
        } else {
            echo "Combustível insuficiente para percorrer $km km!<br>";
        }
    }

    public function exibirInfo() {
        echo "Modelo: {$this->modelo} | Consumo: {$this->consumo} km/L | Tanque: " .
             number_format($this->tanque, 2, ',', '.') . " L<br>";
    }
}
$carro = new Carro("Honda Civic", 12, 20); // 12 km/L, tanque inicial de 20 L
$carro->exibirInfo();
$carro->dirigir(60); // deve consumir 5 L
$carro->abastecer(10);
$carro->exibirInfo();
```

🔎 **Observações:**

- **Comentem o código explicando cada linha importante.**
- **Envie uma print de cada atividade**